

Data Formats & Storage

Week 5 • choosing representations that preserve meaning, portability, and reproducibility

CSV

EXCEL

JSON

XML

DATABASE

A data format is not just packaging. It determines what structure can be expressed — and what can be lost.

Today's roadmap

From simple tables to storage decisions that scale

- 1 **Why format matters**
- 2 **Tabular formats**
- 3 **Hierarchical formats**
- 4 **Data types & hazards**
- 5 **Storage & databases**
- 6 **Hands-on + Homework 5**

● Week 5 question

What representation and storage approach best preserves the meaning of the data while supporting the people, tools, and analyses that need to use it?

● Preview of Week 6

When files stop being enough, we move to relational databases and SQL.

Bridge from Week 4

Documentation tells the story; format carries the structure

WEEK 4 → WEEK 5

● Week 4

Provenance, metadata, data dictionaries, README files, and reproducibility.

● Week 5

How the values and relationships are represented: flat tables, workbooks, nested objects, tagged documents, or databases.

● Together

A well-documented dataset in a poor format can still be hard to use. A technically elegant format without documentation can still be ambiguous.

Meaning = values + structure + documentation.

Why does data format matter?

- Controls what structures can be represented naturally.
- Affects how easily different software can read the data.
- Can preserve or silently destroy identifiers, dates, missing values, and precision.
- Changes how well files work with version control and reproducible workflows.
- Influences scale, collaboration, validation, and long-term reuse.

● Same biology

The scientific meaning might be identical.

● Different representation

The same records can be stored as CSV rows, Excel cells, JSON objects, or database tables.

● Different risks

Each representation creates different opportunities for errors and different strengths.

Same data, different containers

Three representations of one patient visit

Flat table

```
participant_id,visit_date,hb1c  
0012,2026-01-08,7.8  
0013,2026-01-09,8.4
```

Hierarchical object

```
{  
  "participant_id": "0012",  
  "visit": {"date": "2026-01-08",  
            "hb1c": 7.8}  
}
```

The “best” format depends on the shape of the data and the workflow — not on which extension looks more modern.

Tabular vs. hierarchical data

Start by recognizing the shape

● Tabular

Rows = observations

Columns = variables

Natural for rectangular analysis datasets.

Examples: CSV, TSV, a simple Excel sheet, relational database table.

● Hierarchical

Objects can contain objects or arrays.

Natural for nested relationships and API responses.

Examples: JSON, XML, FHIR resources, configuration files.

Key question Can the structure be expressed cleanly without repeated, ambiguous, or duplicated information?

CSV: comma-separated values

Simple, portable, and intentionally limited

```
participant_id,visit_date,age,sex,hba1c  
0012,2026-01-08,54,F,7.8  
0013,2026-01-09,61,M,8.4  
0018,2026-01-16,44,F,
```

● Header row

Names the fields / variables.

● Rows

Each line is typically one observation.

● Plain text

Readable without specialized software.

● Portable

Works across R, Python, SAS, Stata, and many other tools.

● Limited schema

The file does not inherently know that a field is an ID, date, category, or unit.

When CSV/TSV works very well

STRENGTHS

● Cross-platform

Most data tools can read and write it.

● Version-control friendly

Plain-text differences are relatively easy to inspect.

● Human inspectable

You can open the raw text and see what is actually stored.

● Good for analysis-ready tables

Excellent when the data are rectangular and modest in size.

CSV is often a good default for a clean analysis table, but not necessarily for the entire data system.

CSV pitfalls are usually semantic, not visual

- No built-in declaration of data types.
- Dates may be ambiguous across locales.
- Leading zeros in identifiers may disappear after import.
- Missing values may be represented inconsistently.
- Commas, quotes, and line breaks inside text fields require proper escaping.
- Units and category meanings live outside the file unless documented.

● **Dangerous assumption**

“It opened correctly in Excel, so the data must be correct.”

● **Better habit**

Inspect types and values after import. Define an explicit schema or data dictionary.

TSV: same idea, different delimiter

Tab-separated values

● What changes?

The separator is a tab rather than a comma.
The conceptual structure is still a flat table.

● Why use it?

Useful when free-text fields contain many commas, or when a tool expects tabs.

● What does not change?

It still lacks rich schema, types, relationships, and embedded metadata.

```
participant_id      visit_date      hb1c
0012      2026-01-08      7.8
```

Excel workbooks: powerful for people, risky as invisible logic

Useful features

- Multiple sheets
- Formulas and calculated cells
- Data validation and dropdowns
- Comments, formatting, filters
- Familiar interface for collaborators

Reproducibility hazards

- Automatic date/number conversion
- Mixed data types in one column
- Hidden sheets/rows/columns
- Formulas can change results silently
- Manual edits can be hard to audit
- Binary diffs are poor in Git

Excel can be an excellent interface. It is not automatically an excellent source of truth.

CSV or Excel? Ask what the file needs to do

Need	CSV / TSV	Excel
Cross-software analysis	Very strong	Good, but import behavior can vary
Human data entry	Weak	Strong when validation is used
Transparent text diffs	Strong	Weak
Multiple sheets / formulas	Not supported	Strong
Long-term master with rich relationships	Poor fit	Usually poor fit, so consider a database

JSON: objects, keys, values, and arrays

A natural representation for nested data and APIs

```
{
  "participant_id": "0012",
  "demographics": {"age": 54, "sex": "F"},
  "visits": [
    {"date": "2026-01-08",
     "labs": [{"name": "HbA1c", "value": 7.8}]}
  ]
}
```

● Strength

Naturally expresses nesting and repeated sub-records.

● Common use

Web APIs, data exchange, configuration, event messages.

● Trade-off

Many statistical workflows eventually flatten selected pieces into tables.

XML: tagged hierarchical text

Verbose, structured, and still important in many systems

```
<patient id="0012">
  <visit date="2026-01-08">
    <lab name="HbA1c" unit="%">7.8</lab>
  </visit>
</patient>
```

● Why it exists

Explicit tags, attributes, namespaces, and schemas support complex standards and validation.

● Where you may encounter it

Legacy clinical systems, standards-based exchange, configuration, document formats.

● Trade-off

More verbose than JSON and less pleasant for quick exploratory analysis.

JSON vs. XML

Both can represent hierarchy; ecosystems differ

Dimension	JSON	XML
Structure	Objects + arrays	Elements + attributes
Typical modern use	APIs / web services	Standards / enterprise / legacy exchange
Verbosity	Usually compact	Usually more verbose
Schemas / validation	Available but ecosystem-dependent	Mature schema languages and namespaces
Analysis workflow	Often parsed then flattened	Often parsed then transformed

Data types are part of the meaning

● Numeric

7.8, 54, 165.1

● Text

"0012", "Memphis"

● Boolean

true / false

● Date/time

2026-01-08 14:30

● Categorical

sex, diagnosis class, site

● Missing

NA / null / blank — but define it

A value can look numeric and still be conceptually text.

Dates and times: deceptively difficult

COMMON PROBLEMS

● Ambiguous text

03/04/2026 — March 4 or April 3?

● Date vs datetime

A visit date is not the same as an event timestamp.

● Time zones

A timestamp without a zone can shift meaning across systems.

For text dates, ISO-style YYYY-MM-DD is usually a safer, unambiguous convention.

2026-01-08

Categorical variables: define the vocabulary

```
sex: F / M  
diabetes_status: Y / N
```

```
sex: Female / Male  
diabetes_status: Yes / No
```

```
sex: 0 / 1  
diabetes_status: 0 / 1
```

All three may be technically readable — but without a data dictionary, 0/1 is ambiguous.

- Define allowed values.
- Define the meaning of each code.
- Keep coding consistent across files and time.
- Do not assume another analyst will guess correctly.

Missing values: one concept, many dangerous representations

blank

NA

NULL

999

unknown

not done

Do not make the analyst infer whether 999 is a real value, a sentinel code, or a data-entry error.

● Better practice

Use explicit missingness supported by the format/tool, and document any special reason codes.

● Important nuance

“Missing,” “not measured,” “not applicable,” and “below detection” may be scientifically different.

Identifiers are labels, not measurements

0012



12

What was lost?

- Leading zeros
- Exact string identity
- Potential linkage compatibility

● Store as text

Participant IDs, medical record surrogates, specimen IDs, ZIP codes, and many codes should not be treated as numeric quantities.

● Validate after import

Check that IDs remain unique, complete, and character-for-character identical.

Keep values numeric; keep units explicit

```
weight  
80.7 kg  
96.2  
71.4 kilograms
```

Problem

- Mixed text and numeric values
- Harder validation
- Harder conversion
- Ambiguous if some rows omit the unit

Prefer `weight_kg = 80.7` and document the unit as metadata.

Text encoding: preserving characters across systems

● What is encoding?

A mapping between characters and bytes. Different systems may interpret the same bytes differently if encoding is unknown.

● Why biomedical data care

Names, locations, symbols, non-English text, and special characters can become corrupted.

● Practical default

UTF-8 is a widely supported choice for portable text data.

Example corruption: "José" → "JosÃ©"

Encoding errors do not always look dramatic — sometimes they quietly break matching and linkage.

A filename extension is not a data-quality guarantee

.CSV

≠

clean data

- A valid CSV can contain inconsistent categories.
- A valid Excel workbook can contain hidden formulas and stale sheets.
- A valid JSON document can use unstable or undocumented keys.
- A valid XML file can conform syntactically but still be scientifically ambiguous.

Format and storage location are separate decisions

● Local workstation

Fast, convenient
Risky for protected data; backup and collaboration depend on setup

● Institutional network / managed drive

Centralized, backed up, access-controlled
Permissions and performance vary

● Cloud collaboration platform

Easy collaboration and versioning
Must be institutionally approved for data sensitivity

● Secure research environment

Designed for restricted/clinical data
May restrict exports, tools, or internet access

A CSV containing protected clinical data is still protected clinical data.

When does a file problem become a database problem?

● Files work well when...

The dataset is modest, mostly rectangular, edited by few people, relationships are simple, and you usually analyze a snapshot from start to finish.

● Consider a database when...

Data span many related entities, grow continuously, are queried repeatedly, need concurrent access, require strong constraints, or are too large/complex for reliable file-based workflows.

Week 6: relational databases turn relationships into first-class structure — and SQL lets us query them.

A reproducible format-selection workflow

- 1 Describe the structure** Flat? nested? multiple related tables?
- 2 Identify users & tools** R, Python, Excel, API, database?
- 3 Define types & schema** IDs, dates, categories, missingness, units.
- 4 Check scale & collaboration** Size, concurrent users, update frequency.
- 5 Apply governance** Where may the data be stored and who may access them?
- 6 Document the choice** Record format, encoding, schema, version, and conversion steps.

Choose the representation: four biomedical scenarios

There can be more than one defensible answer — justify the trade-offs

● Scenario A

A 5,000-row analysis table must be shared with collaborators using R, Python, and SAS.

● Scenario B

A web API returns one patient with repeated visits and several lab arrays per visit.

● Scenario C

Two coordinators maintain a screening log and need dropdown validation and quick review.

● Scenario D

A hospital warehouse contains millions of patients, encounters, diagnoses, medications, and labs.

Discuss: format + storage + documentation

Hands-on lab: inspect the same data in multiple formats

FILES IN THE WEEK 5 PACKAGE

- patient_visits.csv
- patient_visits.tsv
- patient_visits.xlsx
- patient_visits.json
- nested_patient_example.json
- messy_formatting.csv

● Task 1

Compare how one record is represented in CSV, XLSX, and JSON.

● Task 2

Identify at least five hazards in messy_formatting.csv.

● Task 3

Decide which format you would use for analysis, exchange, and long-term storage — and explain why.

Homework 5 — Choosing Data Formats Deliberately

Part 1

Compare CSV, Excel, and JSON representations

25 pts

Part 2

Diagnose at least five formatting problems

25 pts

Part 3

Choose formats for four biomedical scenarios

25 pts

Part 4

Import/export round trip and verify what changed

20 pts

Synthesis

Questions to ask before trusting an Excel “main raw dataset”

5 pts

Estimated effort: 60–90 minutes. The goal is reasoning, not memorizing extensions.

Takeaways

● Structure first

Choose a representation that matches the shape of the data.

● Types matter

IDs, dates, categories, missingness, units, and encoding are part of the meaning.

● Convenience has trade-offs

Human-friendly spreadsheets can hide logic and silent conversions.

● Storage is governance

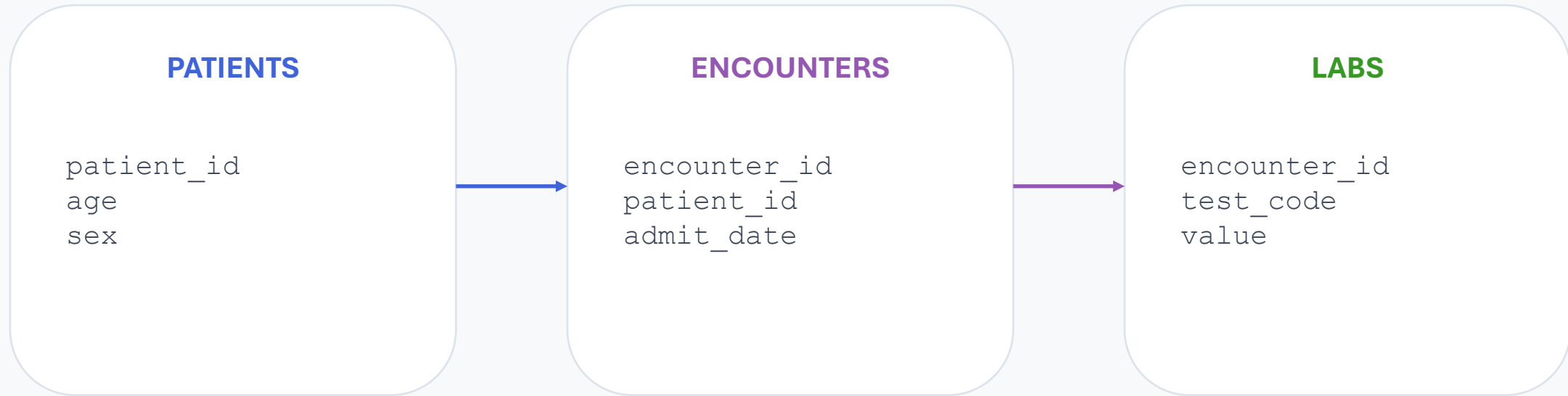
Where the file lives is separate from the file format.

● Scale changes the tool

Complex linked data eventually require database structure.

Next week: relational databases & SQL

When separate files are no longer the right abstraction



Week 6 question: how do we represent these relationships efficiently and query only the data we need?